

LEWIS UNIVERSITY

Final Project Report

AI-Driven Microservices Fitness Coaching Platform

Semester/Year

By

Samba Siva Rao Komeriseti

Rishitha Musunuru

Reeshmitha Reddy Gaddam

Department of Computer and Mathematical Sciences

Submitted in partial fulfillment of the requirements

for the final project of
CPSC-69100 (Computer Science Master's Project)
Term: Spring 2026

Concentration in Software Engineering

Advisor

Dr. Fadi Wedyan

Abstract

Fitness Coaching is an AI-enhanced fitness tracking platform designed to help users monitor their physical activities and receive personalized workout insights. Built using a modern microservices architecture, the system integrates Spring Boot services, Apache Kafka for event-driven processing, PostgreSQL and MongoDB for hybrid data storage, and Keycloak for secure authentication. A React-based frontend enables users to log activities while the AI Service analyzes these logs using Google Gemini to generate personalized recommendations, improvements, and behavioral insights. The platform emphasizes scalability, reliability, and ethical data handling, ensuring user privacy and safe AI usage. Throughout development, software engineering principles—including system modeling, Agile iteration, distributed system design, and secure coding practices—were applied. The result is a robust, cloud-ready application that demonstrates how artificial intelligence and microservices can work together to enhance user wellness and provide meaningful, personalized fitness guidance.

Contents

Abstract	2
1. Theory and Background.....	6
1.1 Introduction to Software Engineering	6
1.2 Software Engineering Ethics	6
1.3 Software Development Lifecycle (SDLC)	7
1.4 Agile Software Development Process	9
2. Introduction to the Project and Client	9
2.1 Purpose of the Project	10
2.2 Client Background.....	10
2.3 Team Objective.....	10
3. Literature Review.....	10
3.1 Growth of Digital Fitness Technologies.....	11
3.2 Microservices in Modern Applications	11
3.3 Event-Driven Architecture (Kafka)	11
3.4 Ethical Considerations in Health Software	11
3.5 AI-Driven Recommendations.....	12
4. Project Approach and Information Gathering.....	12
4.1 Understanding the Client and Project Needs.....	12
4.2 Understanding the Users	13
4.3 Physical and Technological Environment	13
4.4 Existing Software and Hardware.....	14
4.5 Budget Considerations	14
4.6 Security Concerns	14
4.7 Ethical Issues.....	15
5. Ethical Issues Encountered and Addressed.....	15
5.1 Data Privacy and Confidentiality	15
5.2 Ethical Use of AI	16

5.3 Fairness and Transparency	16
5.4 Secure Authentication and Authorization.....	16
5.5 Respectful User Interaction.....	17
6. System Design and Architecture	17
6.1 High-Level Architecture Overview	17
6.2 Event-Driven Design with Kafka.....	18
6.3 Databases.....	18
6.4 UML Diagrams Used	19
7. Implementation Details.....	22
7.1 Backend Implementation (Spring Boot Microservices).....	22
7.2 Frontend Implementation (React + Vite + Redux Toolkit)	23
7.3 Authentication Implementation (Keycloak).....	24
7.4 Messaging with Kafka.....	24
7.5 Deployment and Runtime Environment.....	24
8. Testing, Security, and Documentation	25
8.1 Testing Strategy	25
8.2 Security Framework	26
8.3 Documentation	26
9. Balancing Business Needs and Budget Constraints.....	27
9.1 Budget Limitations.....	27
9.2 Business Needs Analysis.....	28
9.3 Balancing Performance and Cost	28
10. Application of Learning Outcomes	29
10.1 Software Engineering Principles.....	29
10.2 Distributed Systems & Microservices.....	29
10.3 Artificial Intelligence & Prompt Engineering	29
10.4 Database Design & Data Modeling.....	30
10.5 Web & Frontend Development.....	30

10.6 Cybersecurity & Authentication.....	30
10.7 DevOps & Deployment.....	31
10.8 Team Collaboration & Communication	31
11. Conclusion	31
APPENDIX.....	33
REFERENCES	39

1. Theory and Background

1.1 Introduction to Software Engineering

Software engineering is the systematic, disciplined, and quantifiable approach to developing and maintaining software systems. It applies engineering principles to ensure that software is reliable, scalable, secure, and meets user needs. As modern systems grow more complex, software engineering practices become essential for managing requirements, architecture, implementation, testing, and evolution.

Fitness Coaching is a practical example of applying software engineering principles to a real-world problem—building a distributed, AI-powered fitness-tracking platform. The project required careful architectural decisions, modular service design, and structured workflows.

Core software engineering principles applied include:

- **Modularity:** Separating functionality into microservices (User, Activity, AI, Gateway).
- **Abstraction:** Hiding internal complexity through APIs and standardized communication.
- **Scalability:** Using microservices, Kafka streaming, and asynchronous processing.
- **Reliability:** Implementing error handling, retries, and fault-tolerant communication.
- **Maintainability:** Using clean architecture, documentation, and standardized patterns.

Fitness Coaching showcases how theoretical engineering concepts support the creation of a large, extensible, cloud-ready application integrating AI, databases, and authentication systems.

1.2 Software Engineering Ethics

Software engineering ethics guide responsible decision-making throughout the development lifecycle, ensuring fairness, transparency, safety, and respect for user privacy. The ACM/IEEE Code of Ethics emphasizes values such as public welfare, confidentiality, integrity, and honesty.

For Fitness Coaching, ethical considerations were highly important because the system involves sensitive fitness and behavioral data. Key ethical issues and actions included:

a. Privacy of User Data

User activities, habits, calorie burns, hydration levels, and workout behaviors are potentially sensitive information. To protect privacy:

- Strong authentication using **Keycloak** and **OAuth2/OIDC** was implemented.
- User identity and tokens are securely validated through the Gateway.
- Sensitive fields are never stored without encryption or access control.

b. Ethical AI Practices

The AI service generates fitness recommendations using Google Gemini. Ethical considerations included:

- Ensuring recommendations are **supportive, safe, and non-judgmental**.
- Avoiding biased or harmful interpretations of user data.
- Making sure AI suggestions are not misleading or medically inappropriate.

c. Security and Responsible Access

Tokens, passwords, and health-related data were handled with strict security:

- No passwords stored in plaintext.
- JWT validation and role-based access control enforced.
- Backend services restrict data access by user ID.

Through these actions, the project aligned with ethical software development practices and protected user trust.

1.3 Software Development Lifecycle (SDLC)

The Software Development Lifecycle provides a structured framework for planning, designing, building, testing, and maintaining software. For this project, a **hybrid SDLC** approach was used, combining traditional phases with Agile iteration.

Phases Followed in Fitness Coaching

1. Requirement Gathering

- Defined user needs: tracking activities, generating AI insights, secure login.
- Identified functional and non-functional requirements.
- Collected instructor feedback and updated scope progressively.

2. System & Architecture Design

- Selected microservices architecture for scalability.
- Designed databases: PostgreSQL for users, MongoDB for activities.
- Created UML diagrams (class diagrams, sequence diagrams, component diagrams).
- Planned event-driven communication using Kafka.

3. Implementation

- Built Spring Boot services with REST APIs.
- Integrated Keycloak authentication.
- Developed Kafka publisher/subscriber pipelines.
- Implemented frontend in React with secure token handling.
- Integrated Google Gemini for AI recommendations.

4. Testing

- Test cases for backend APIs.
- Kafka message flow verification.
- Manual and UI testing for React components.
- Validation of AI accuracy and response parsing.

5. Deployment

- All services containerized using Docker.
- Configured service discovery and centralized configuration.
- Prepared environment for final demonstration.

6. Maintenance & Evolution

Throughout the semester, several improvements were made:

- Added more activity metrics (hydration, meal timing, time-of-day).
- Enhanced AI prompt structure for better recommendations.
- Improved frontend user experience (edit/delete workflows).
- Implemented error handling and retry logic in Kafka listeners.

The hybrid SDLC allowed the team to maintain structure while remaining flexible enough to incorporate feedback, new features, and evolving project requirements.

1.4 Agile Software Development Process

Agile development emphasizes continuous iteration, teamwork, and rapid adaptation. This project followed Agile principles through:

- **Weekly sprints** aligned with Milestone deliverables.
- **Team roles and task distribution** using GitHub.
- **Frequent reflection and updates** based on instructor feedback.
- **Incremental delivery:**
 - Milestone 1: Core microservices, DB setup, authentication
 - Milestone 2: Kafka + AI integration+ Full UI
 - Milestone 3: recommendations + polishing

Agile allowed the group to gradually refine the system and adapt to changing needs during development.

2. Introduction to the Project and Client

Fitness Coaching is an AI-enhanced fitness tracking platform designed to help users improve their workout habits and health outcomes. Traditional fitness applications often focus only on recording activities without offering deeper insight or personalized recommendations. Fitness Coaching addresses this gap by integrating AI-generated analysis of user behavior.

2.1 Purpose of the Project

The system aims to:

- Provide secure fitness activity logging
- Analyze user behavior patterns
- Deliver personalized AI suggestions
- Offer an intuitive and responsive user interface

2.2 Client Background

The “client” for this capstone project represents:

- Individual users who wish to understand their fitness habits
- Trainers or wellness professionals who need personalized insights
- Fitness-focused organizations exploring AI-driven solutions

Although abstract, the client scenario reflects real-world stakeholders in the digital fitness market.

2.3 Team Objective

The goal was to build a fully functioning full-stack system demonstrating:

- Distributed systems
- Secure authentication
- Event-driven processing
- AI integration
- Professional UI/UX
- Ethical and secure handling of user data

This introduction sets the stage for the technical, ethical, and practical components described throughout the report.

3. Literature Review

The literature review explores prior research, industry trends, and supporting frameworks relevant to the project.

3.1 Growth of Digital Fitness Technologies

Research indicates increasing adoption of digital fitness platforms and wearable devices.

Studies show:

- Users are more likely to maintain an exercise routine when receiving personalized feedback.
- AI-enhanced interventions improve consistency and motivation.
- Behavioral analytics can predict user engagement trends.

This supports the integration of AI-based insights in Fitness Coaching.

3.2 Microservices in Modern Applications

Academic and industry sources highlight microservices as:

- Highly scalable
- Independent and modular
- Easier to deploy and maintain
- Ideal for event-driven and cloud-native applications

This justified splitting the project into separate services (Activity, User, AI, Gateway).

3.3 Event-Driven Architecture (Kafka)

Kafka is widely adopted for:

- High throughput communication
- Decoupled, asynchronous data flow
- Real-time event processing

Literature confirms its role in scalable analytics pipelines—aligning with the AI recommendation flow.

3.4 Ethical Considerations in Health Software

Sources emphasize:

- Protecting user health data
- Transparent and fair AI models
- Respectful user feedback
- Avoiding harmful or prescriptive language

This guided the ethical AI and security features of the project.

3.5 AI-Driven Recommendations

Research on machine learning in fitness shows:

- Personalized suggestions improve health outcomes
- Generative AI can interpret structured data effectively
- Prompt engineering improves model accuracy

Gemini's structured output capabilities made it suitable for delivering fitness insights.

4. Project Approach and Information Gathering

The development of Fitness Coaching followed a systematic, research-driven, and iterative approach. Before beginning implementation, the team spent time understanding the project context, studying user needs, defining constraints, and analyzing the technological landscape. This allowed the system to be built with a clear vision and informed engineering decisions.

4.1 Understanding the Client and Project Needs

The “client” in this capstone scenario represents individuals or organizations interested in tracking fitness activities and receiving meaningful insights from their data. To address these expectations, the team gathered real-world references from existing fitness trackers such as Google Fit, Strava, and MyFitnessPal. Common limitations identified across these platforms included:

- Limited personalization without subscription paywalls
- Lack of detailed behavior-based recommendations

- Minimal integration with AI for analysis
- Systems that record data but do not interpret it

Through this analysis, the team defined the primary objective: **to build a free, AI-enhanced fitness platform that gives users actionable insights and not just raw data.**

4.2 Understanding the Users

Two primary user types were identified:

1. General Users

Individuals who want to:

- Log their daily workouts
- Understand how their habits affect performance
- Improve consistency
- Receive AI-driven suggestions

2. Administrators / Trainers (Future Expansion)

Stakeholders who may want to:

- Monitor aggregated performance metrics
- Customize training programs
- Provide targeted coaching

Although the initial build focuses on individual users, the architecture supports future expansion for trainer dashboards.

4.3 Physical and Technological Environment

Users interact with Fitness Coaching through a browser-based interface. Requirements included:

- Accessibility from mobile and desktop
- Low latency for activity updates

- Rapid loading of recommendations
- Secure token handling in browser storage

The backend was designed for cloud deployment using Docker containers to ensure platform independence.

4.4 Existing Software and Hardware

Prior to development, the team evaluated existing systems to avoid reinventing the wheel:

- **Authentication:** Keycloak (industry-standard open-source IAM)
- **Databases:** PostgreSQL + MongoDB
- **Messaging System:** Kafka for scalable event-driven communication
- **AI Models:** Google Gemini due to strong structured output capabilities
- **Frontend Framework:** React + Vite for a fast, modern UI
- **Backend Framework:** Spring Boot microservices

These technologies were chosen because they are open-source, reliable, and widely used in modern enterprise architectures.

4.5 Budget Considerations

Since this was an academic project, the priority was to minimize costs. Strategies included:

- Using entirely **open-source tools** (Kafka, Keycloak, PostgreSQL, MongoDB)
- Leveraging **free-tier AI usage** with Gemini
- Running services locally using **Docker Compose**
- Avoiding proprietary SDKs or cloud services

This ensured the system could be built and demonstrated without financial overhead.

4.6 Security Concerns

Key issues identified:

- Unauthorized access to sensitive fitness data
- Token tampering

- Improper AI feedback leading to harm
- Potential injection attacks

Mitigation strategies:

- Strict OAuth2/OIDC token validation
- Input validation on backend
- Secure communication via HTTPS-ready configurations
- Ethical AI wording
- Role-based access control

4.7 Ethical Issues

Ethics were integrated throughout the approach:

- No personally identifiable information beyond email is required
- AI recommendations avoid medical claims
- Prompts avoid judgmental language
- Users have full control over their data

This ensured compliance with good engineering ethics and privacy standards.

5. Ethical Issues Encountered and Addressed

Due to the system's nature managing activity data and producing AI-based recommendations ethical considerations were central to development. Several challenges surfaced that required deliberate solutions.

5.1 Data Privacy and Confidentiality

Fitness data can reveal personal health patterns. Ethical concerns included:

- What information should be collected?
- How should it be stored?
- Who can access it?

Actions Taken

- Only authenticated users can access activity data.
- JWT tokens are validated at the Gateway before forwarding any request.
- Sensitive activity data is stored in MongoDB with strict per-user filtering.
- No personal health details outside activity metrics were collected.
- Passwords are NOT stored — Keycloak manages them securely.

5.2 Ethical Use of AI

AI-generated fitness suggestions can influence user behavior. Risks include incorrect or harmful advice.

Mitigation Strategies

- AI prompts ensure suggestions are non-medical, supportive, and risk-free.
- Recommendations emphasize safe practices and general wellness tips.
- AI output is reviewed against ethical AI guidelines.
- No predictions or diagnoses are generated.

The system aims to *motivate* users, not prescribe professional health advice.

5.3 Fairness and Transparency

AI systems may unintentionally generate biased or unclear outputs.

Solutions

- Prompt engineering ensures that inputs are neutral and factual.
- Outputs are structured (JSON) to avoid ambiguous or misleading content.
- No sensitive personal traits influence AI recommendations.

5.4 Secure Authentication and Authorization

Unauthorized access could expose user activity history.

Security Enhancements

- OAuth2 Authorization Code with PKCE
- Role-based access model

- Token expiration and refresh handling
- Strict backend validation (userId must match token subject)

5.5 Respectful User Interaction

The application ensures:

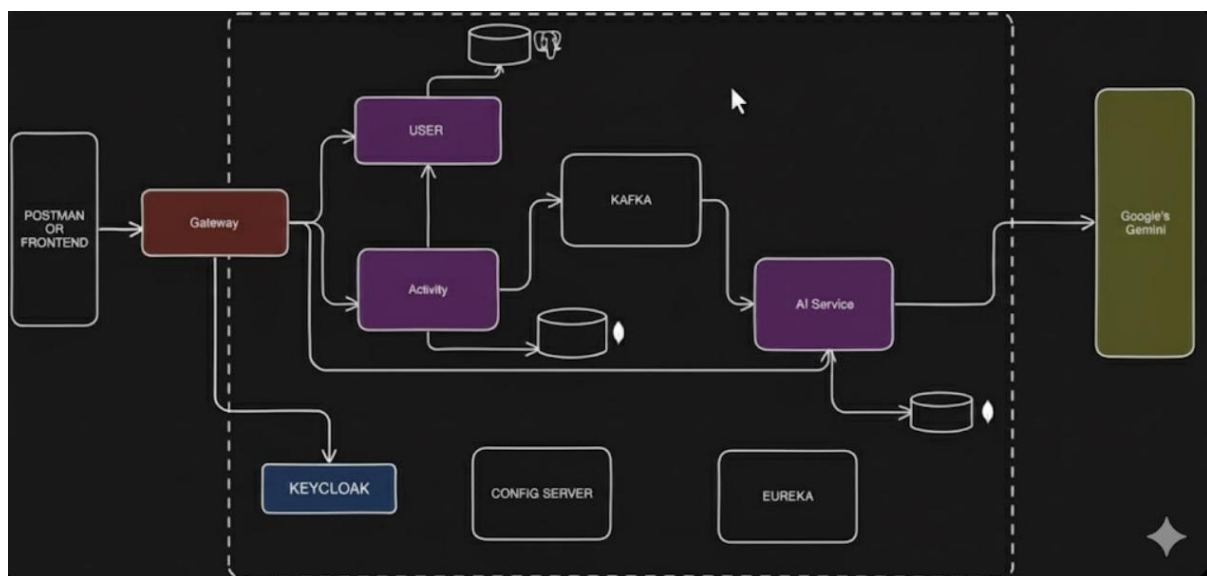
- No shaming language
- No unnecessary comparisons to others
- Positive reinforcement through recommendations

Ethical considerations shaped both the technical and social aspects of Fitness Coaching.

6. System Design and Architecture

Fitness Coaching was designed using a **modern distributed architecture** focused on scalability, modularity, and maintainability. The system is composed of independent microservices that communicate through secure REST APIs and event-driven mechanisms.

6.1 High-Level Architecture Overview



The system contains four major backend microservices:

1. Gateway Service

- Entry point for all client requests
- Validates OAuth2 tokens via Keycloak

- Routes traffic to appropriate services
- Handles user registration and Keycloak synchronization

2. User Service

- Stores user metadata in PostgreSQL
- Provides user lookup and validation
- Syncs with Keycloak during registration

3. Activity Service

- Manages CRUD operations for user activities
- Stores activities in MongoDB
- Publishes Kafka events on create/delete

4. AI Service

- Listens to Kafka events
- Fetches activity data and calls Google Gemini
- Generates and stores recommendations in MongoDB

6.2 Event-Driven Design with Kafka

Kafka ensures asynchronous communication:

- Activities generate events
- AI Service consumes these events independently
- User experience remains smooth and non-blocking

This architecture allows scaling services horizontally.

6.3 Databases

PostgreSQL

- Structured user metadata
- Strong relational constraints

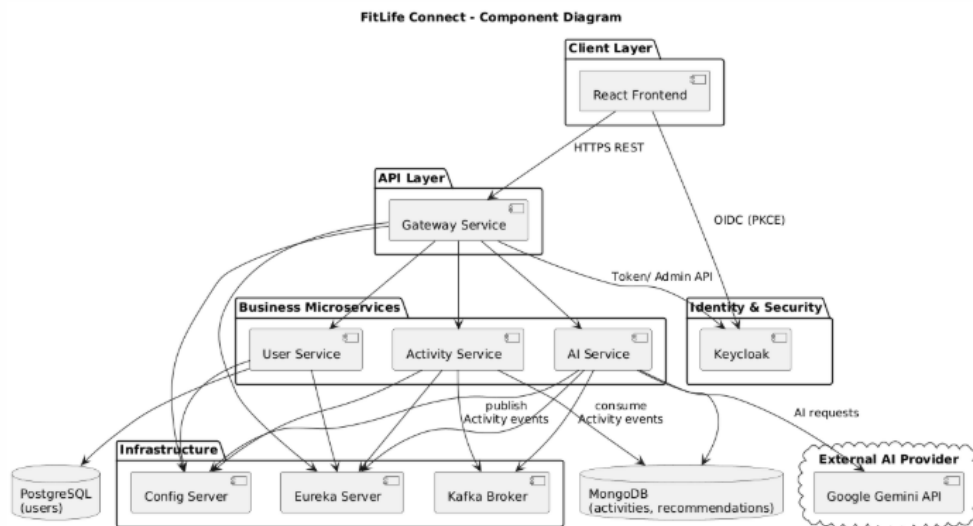
MongoDB

- Flexible documents for activities and AI recommendations
- Suits dynamic AI output structures

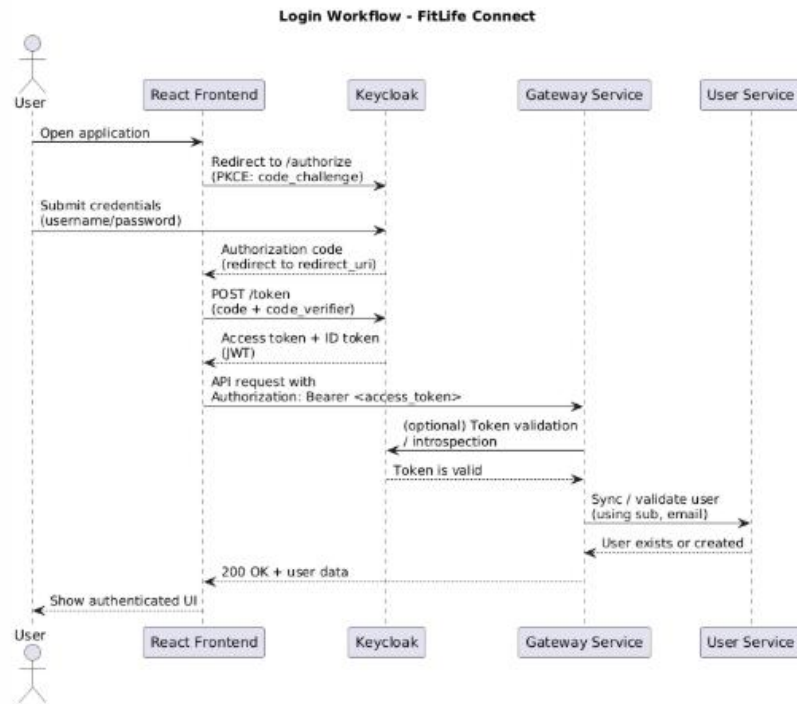
6.4 UML Diagrams Used

As part of system modeling:

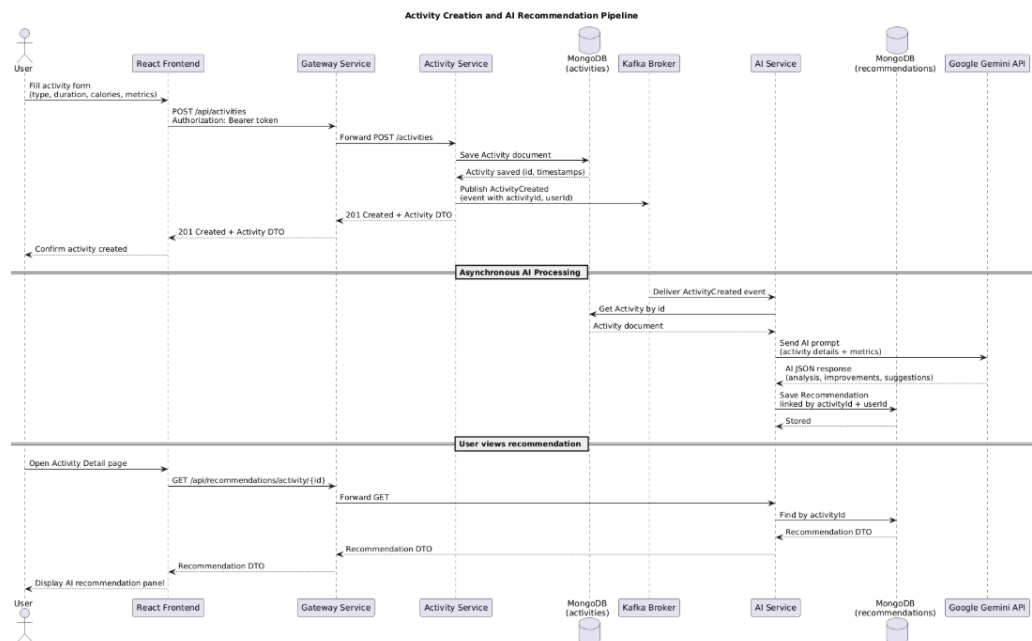
- **Component Diagram:** Shows microservices and external dependencies.



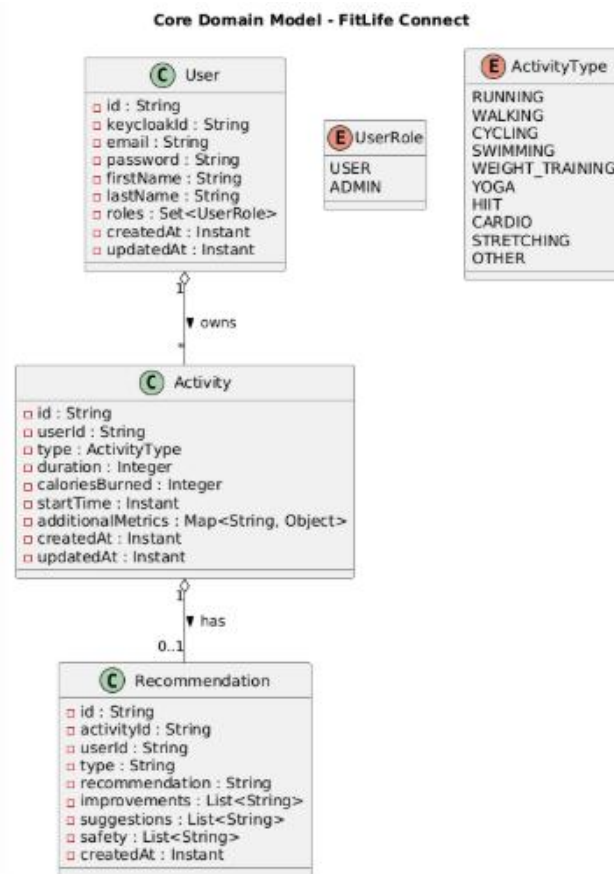
- **Sequence Diagrams:**
 - Login workflow



- Activity creation → AI pipeline



- **Class Diagrams:** For User, Activity, Recommendation models



These diagrams helped clarify responsibilities and interactions before coding.

6.5 Technology Stack

Layer	Technology
Frontend	React, Vite, Redux Toolkit
Auth	Keycloak (OAuth2 + OIDC PKCE)
Backend	Spring Boot 3 Microservices
Messaging	Apache Kafka
Databases	PostgreSQL + MongoDB
AI Integration	Google Gemini API
Deployment	Docker, Docker Compose
Discovery	Eureka Server

Configuration	Config Server
---------------	---------------

6.6 Architectural Strengths

- High scalability
- Fault tolerance
- Loose coupling of components
- Easy maintainability
- Secure access control
- Cloud-ready architecture

7. Implementation Details

The implementation of Fitness Coaching required coordinating multiple technologies across backend, frontend, authentication, messaging, and AI components. The system was constructed using modern engineering practices, GitHub version control, and containerized runtime environments to ensure consistency and scalability.

7.1 Backend Implementation (Spring Boot Microservices)

The backend consists of four independent Spring Boot microservices—Gateway Service, User Service, Activity Service, and AI Service. Each microservice follows clean architecture principles, encapsulating its domain logic, repository layer, controller layer, and DTO mapping.

7.1.1 Gateway Service

- Handles authentication and routing.
- Validates JWT access tokens using Keycloak’s public keys.
- Provides a secure user registration endpoint.
- Forwards all valid requests to appropriate microservices using Spring Cloud Gateway.

7.1.2 User Service

- Uses PostgreSQL to store user metadata.
- Syncs user details during registration triggered via Gateway.

- Provides APIs for retrieving and validating user information.
- Uses JPA and Hibernate for ORM and schema management.

7.1.3 Activity Service

- Stores user activities in MongoDB, enabling flexible JSON-like documents.
- Contains CRUD operations for fitness activities.
- Publishes Kafka events (activity_created, activity_deleted) after relevant operations.
- Includes validation for user identity (userId must match token subject).

7.1.4 AI Service

- Subscribes to Kafka topics to receive activity events asynchronously.
- Retrieves the activity document from MongoDB.
- Constructs structured AI prompts containing:
 - Activity type
 - Duration
 - Calories burned
 - Hydration
 - Time of day
 - Meal timing
- Sends prompt to Google Gemini API and retrieves structured JSON output.
- Stores AI recommendations in MongoDB linked by activityId.

7.2 Frontend Implementation (React + Vite + Redux Toolkit)

The frontend is built as a modern single-page application (SPA) using React functional components with hooks.

Key Components:

- **SignupModal.jsx** – Registers new users via Gateway API
- **ActivityForm.jsx** – Accepts activity inputs and additional metrics

- **ActivityList.jsx** – Displays recent activities
- **ActivityDetail.jsx** – Shows activity details + AI recommendations
- **CompleteRecommendation.jsx** – Aggregated view of all advice
- **Navbar.jsx** – Provides navigation + token-based logout

State Management (Redux Toolkit)

- Stores access token, user info, and authentication state.
- Ensures protected routes require valid tokens.

API Communication

All API requests are:

- Sent through api.js
- Passed with Authorization: Bearer <token> header
- Routed through Gateway to apply security filters

7.3 Authentication Implementation (Keycloak)

- Deployed as a Docker container locally.
- Configured with required clients (frontend, gateway).
- Uses Authorization Code Flow with PKCE for enhanced browser security.
- User registration triggers an admin API call to create the account in Keycloak.

7.4 Messaging with Kafka

- Kafka and Zookeeper deployed using Docker Compose.
- Activity events published from Activity Service are consumed by AI Service listeners.
- Retry logic prevents failures if Gemini API temporarily fails.

7.5 Deployment and Runtime Environment

All services were containerized using Docker for consistent deployment:

- docker-compose.yml orchestrates multi-container setup
- Ensures all services, Kafka, Keycloak, and databases can run locally

This guarantees a standardized development and testing environment.

8. Testing, Security, and Documentation

8.1 Testing Strategy

Testing was performed at multiple layers to ensure the robustness of the system.

8.1.1 Unit Testing

Each microservice includes unit tests for:

- Controllers
- Services
- Repositories

JUnit and Mockito were used for mocking and validating logic.

8.1.2 Integration Testing

Integration tests verified:

- REST API communication flows
- Kafka message publishing and consumption
- MongoDB and PostgreSQL database interactions
- Token authentication across services

8.1.3 Frontend Testing

Manual testing was performed to check:

- Form validation
- JWT expiration and logout handling
- Recommendation refreshes after editing or deleting an activity
- UI responsiveness and error handling

8.1.4 AI Output Testing

AI responses were validated to ensure:

- JSON formatting matched expected structure

- Recommendations were safe, ethical, and helpful
- Edge cases did not produce harmful suggestions

8.2 Security Framework

8.2.1 Authentication Security

- Keycloak ensures secure OAuth2/OIDC authentication.
- PKCE prevents authorization code interception.
- Short-lived access tokens + refresh tokens maintain session safety.

8.2.2 Authorization Controls

- Backend validates user IDs against token subjects.
- Activity access is strictly user-scoped.
- Gateway rejects unauthorized or malformed tokens.

8.2.3 Data Security

- No plaintext credentials stored.
- Sensitive activity data restricted per user.
- HTTPS-ready configuration ensures secure communication.

8.2.4 API Security

- All services behind Gateway
- No direct access to internal microservice ports
- CORS restrictions applied

8.3 Documentation

Throughout development, documentation was maintained:

API Documentation

Each microservice includes:

- Endpoint definitions
- Payload specifications

- Success/error response formats

Architecture Documentation

Includes:

- Component diagrams
- Sequence diagrams
- Deployment architecture
- Data flow diagrams

User Documentation

Prepared for:

- Setting up environment
- Running Docker containers
- Registering and using the application

9. Balancing Business Needs and Budget Constraints

One of the project's goals was to design a solution that is not only technically effective but also economically feasible. Building a fitness tracking and AI recommendation platform could require significant infrastructure if not carefully planned.

9.1 Budget Limitations

As a student capstone project, financial resources were limited. The design needed to avoid monthly subscription costs and expensive cloud services.

Cost-Reduction Strategies:

- Leveraged **open-source tools** like Kafka, Keycloak, PostgreSQL, MongoDB.
- Used **Google Gemini free-tier API** for AI analysis.
- Ran microservices in **Docker containers** to avoid cloud hosting costs.
- Chose **React + Vite** (open source) for frontend development.

This resulted in a fully functional system at zero operational cost during development.

9.2 Business Needs Analysis

The client (fitness enthusiasts, trainers, or wellness organizations) would need:

- Secure and private storage of activity logs
- Scalable system for high user volume
- AI-driven insights for personalized motivation
- Easy-to-use interface

The architecture supports these needs through:

- Microservices for scalability
- AI Service for intelligent insights
- Gateway + Keycloak for secure authentication
- Low-cost, maintainable infrastructure

9.3 Balancing Performance and Cost

Performance considerations included:

- Asynchronous AI processing to avoid blocking user actions
- Efficient storage of documents in MongoDB
- Load balancing via Eureka if scaled to production
- Using local Kafka brokers instead of cloud stream processing

These decisions allowed the platform to remain cost-effective while capable of scaling if deployed commercially.

9.4 Long-Term Sustainability

The architecture allows:

- Low hosting costs on cloud platforms (DigitalOcean, AWS Lightsail, etc.)
- Independent microservice upgrades without downtime
- Minimal overhead due to lightweight containerized deployment

Overall, the system achieves the balance between **technical excellence**, **business value**, and **cost efficiency**.

10. Application of Learning Outcomes

This project represents the culmination of multiple program learning outcomes, each applied directly to the design, development, and delivery of Fitness Coaching. The project integrates concepts from software engineering, databases, artificial intelligence, distributed systems, cybersecurity, and full-stack development. Below are the key learning outcomes demonstrated:

10.1 Software Engineering Principles

The project required structured planning, requirement gathering, architectural modeling, iterative implementation, and continuous improvement. Concepts applied include:

- Modularity through microservices
- System modeling using UML diagrams
- SDLC phases implemented in order
- Agile project management with weekly milestones
- Ethical application of engineering practices

10.2 Distributed Systems & Microservices

Students applied distributed systems knowledge by designing:

- Independent Spring Boot microservices
- Load-balanced communication via Eureka
- Centralized configuration management
- Asynchronous communication using Kafka

These concepts align with industry-standard architectures used in scalable enterprise systems.

10.3 Artificial Intelligence & Prompt Engineering

The AI service integrated Google Gemini to generate structured fitness recommendations.

Learning outcomes applied include:

- Constructing optimal prompts
- Understanding generative AI output behavior
- Parsing structured JSON responses
- Validating responses for correctness and safety
- Ensuring ethical AI usage

10.4 Database Design & Data Modeling

Students used:

- PostgreSQL for structured relational data
- MongoDB for flexible NoSQL storage of activity logs and AI results

Outcome-based skills demonstrated:

- Creating schemas for real-world applications
- Designing entity relationships
- Ensuring data integrity and efficient retrieval

10.5 Web & Frontend Development

The frontend applied major web development concepts:

- Building dynamic interfaces with React
- Managing global state with Redux Toolkit
- Implementing secure authentication flows (PKCE)
- Working with REST APIs and error handling

These skills reflect modern UI engineering practices.

10.6 Cybersecurity & Authentication

Security concepts from coursework were integrated through:

- OAuth2 and OpenID Connect
- Secure token flow using Authorization Code + PKCE
- Role-based access control

- Prevention of data exposure through Gateway filtering

Students gained experience implementing real-world enterprise authentication systems.

10.7 DevOps & Deployment

Through Docker and containerization, the project demonstrated:

- Multi-container orchestration
- Environment configuration
- Service isolation and reproducibility

These DevOps skills are directly applicable to cloud deployment environments.

10.8 Team Collaboration & Communication

Soft skills reinforced include:

- Coordinating development tasks
- Version control collaboration on GitHub
- Integrating multiple components built by different team members
- Incorporating instructor feedback into revisions

These outcomes reflect workplace-ready collaboration capabilities.

11. Conclusion

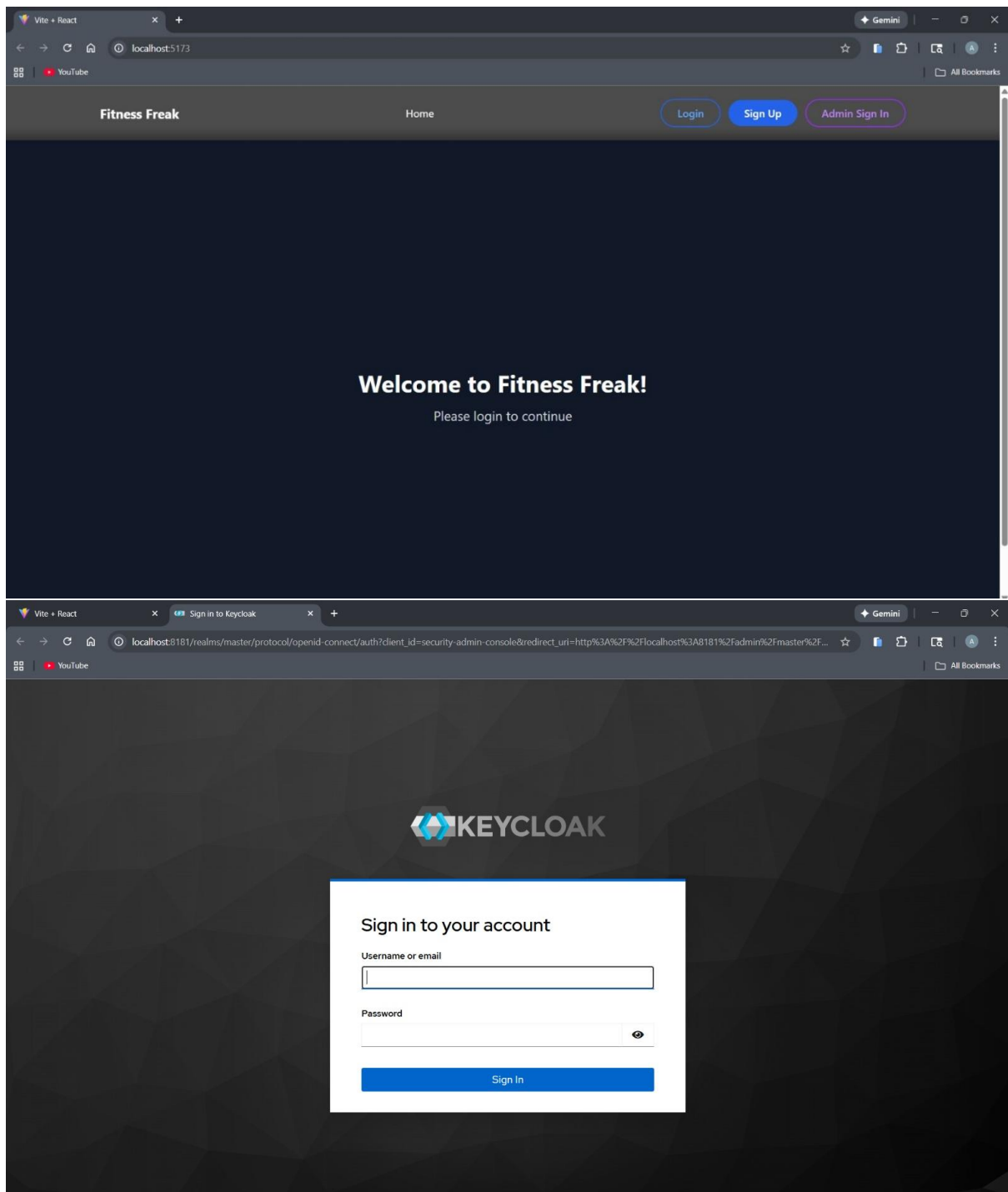
Fitness Coaching represents a fully integrated AI-driven fitness tracking platform built on modern engineering foundations. The system successfully combines microservices, event streaming, secure authentication, user-friendly interfaces, and intelligent recommendations. Through this capstone project, the team demonstrated mastery in software engineering principles, architectural design, ethical analysis, technical implementation, and cross-disciplinary integration.

The platform accomplishes several key objectives:

1. **Secure and Scalable Architecture** The distributed microservices architecture ensures long-term scalability and maintainability. Components operate independently while consistently communicating through secure gateways and Kafka streams.
2. **Rich, Detailed Activity Tracking** Users can record duration, calories, time of day, hydration, and meal timing, allowing for deep behavioral analysis.
3. **AI-Enhanced Fitness Insights** The system interprets activity patterns using Google Gemini, generating personalized recommendations anchored in safe, ethical fitness guidance.
4. **User-Centered Interface** The React frontend provides a seamless experience for logging activities, viewing AI insights, and managing accounts.
5. **Ethically Responsible Engineering** Throughout development, the team adhered to ACM/IEEE ethical standards, protecting user privacy, ensuring secure data handling, and providing non-harmful AI feedback.

The successful completion of Fitness Coaching highlights the team's ability to apply theoretical knowledge to practical, real-world challenges. The platform stands as a professional, extensible foundation for future enhancements such as wearable integration, nutrition analytics, or predictive modeling. More importantly, it demonstrates the team's readiness to design and implement advanced software systems in professional environments.

APPENDIX



The top screenshot shows the Keycloak Administration Console for the 'fitness-app' realm. The browser address bar indicates the URL is `localhost:8181/admin/master/console/#/fitness-app`. A yellow warning banner at the top states: 'You are logged in as a temporary admin user. To harden security, create a permanent admin account and delete the temporary one.' The left sidebar contains the following menu items: 'fitness-app' (Current realm), 'Manage realms', 'Manage', 'Clients', 'Client scopes', 'Realm roles', 'Users', 'Groups', 'Sessions', 'Events', 'Configure', 'Realm settings', 'Authentication', 'Identity providers', and 'User federation'. The main content area displays the Keycloak logo and the text: 'Welcome to fitness-app'. Below this, it says: 'If you want to leave this page and manage this realm, please click the corresponding menu items in the left navigation bar.'

The bottom screenshot shows the 'Create Account' modal form. The browser address bar indicates the URL is `localhost:5173`. The modal has a title 'Create Account' and a subtitle 'Join Fitness Freak today!'. It contains the following fields: 'First Name' (with the value 'John'), 'Last Name' (with the value 'Doe'), 'Email' (with the value 'john@example.com'), and 'Password' (with masked characters '*****'). Below the password field, there is a note: 'Must be at least 8 characters'. At the bottom of the modal is a blue 'Sign Up' button. Below the button, it says: 'Already have an account? [Login here](#)'.

Vite + React

localhost:5173

Gemini

YouTube

All Bookmarks

Fitness Freak

HomeAll ActivitiesGet Recommendations

av@gmail.comLogout

Track Your Activities

Activity Type *

Running

Duration (Minutes) *

Enter duration in minutes

Calories Burned *

Enter calories burned

Date & Time (when activity started) *

11-12-2025 11:01

You can select past date/time; future times are not allowed.

Additional Details (for better AI recommendations)

When did you do this activity?

Select time of day

Was this before or after eating?

Select option

Water drank before/during activity (ml)

e.g., 250, 500

Heart Rate (BPM)

e.g., 120, 150

Distance (km)

e.g., 5, 10.5

Was this before or after eating?

Select option

Water drank before/during activity (ml)

e.g., 250, 500

Heart Rate (BPM)

e.g., 120, 150

Distance (km)

e.g., 5, 10.5

Add Activity

* Please fill in all required fields to add an activity

Recent Activities

CYCLING

🕒 Duration: 100 min

🔥 Calories: 800 kcal

🕒 Time: Morning

💧 Water: 500 ml

View Details →

SWIMMING

🕒 Duration: 30 min

🔥 Calories: 250 kcal

🕒 Time: Morning

💧 Water: 400 ml

View Details →

RUNNING

🕒 Duration: 100 min

🔥 Calories: 800 kcal

🕒 Time: Afternoon

💧 Water: 500 ml

View Details →

See All Activities (4)

The image displays two screenshots of a web application named "Fitness Freak" running on a local host. The browser's address bar shows the URL `localhost:5173/activities/6938345d7ce2ae4c19021185`. The application interface is dark-themed and includes a navigation bar with links for Home, All Activities, and Get Recommendations, along with a search icon and a user profile icon.

Top Screenshot: Activity Details

The "Activity Details" section displays the following information:

- Activity Type:** CYCLING
- Duration:** 100 minutes
- Calories Burned:** 800 kcal
- Date:** 12/9/2025, 8:37:00 AM
- Time of Day:** Morning
- Around Lunch:** BEFORE_EATING
- Water Intake:** 500 ml
- Heart Rate:** 120 BPM
- Distance:** 10 km

At the bottom of the details section are two buttons: "Edit Activity" (yellow) and "Delete Activity" (red).

Bottom Screenshot: AI Recommendation

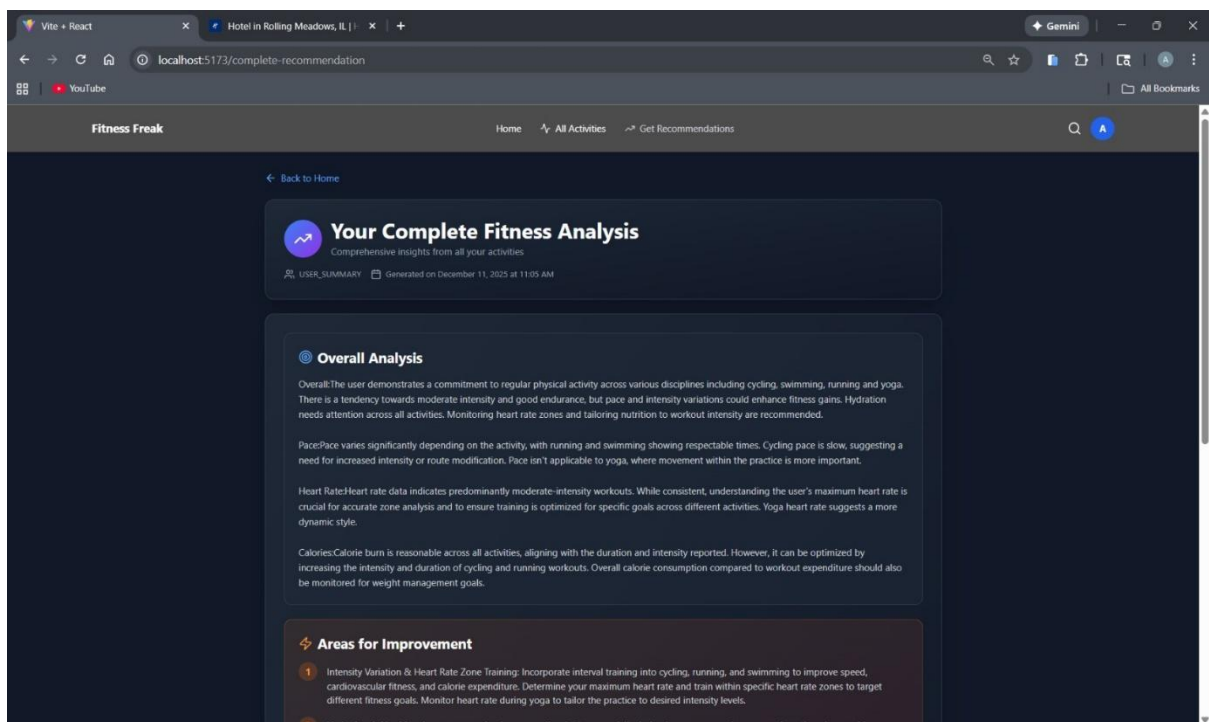
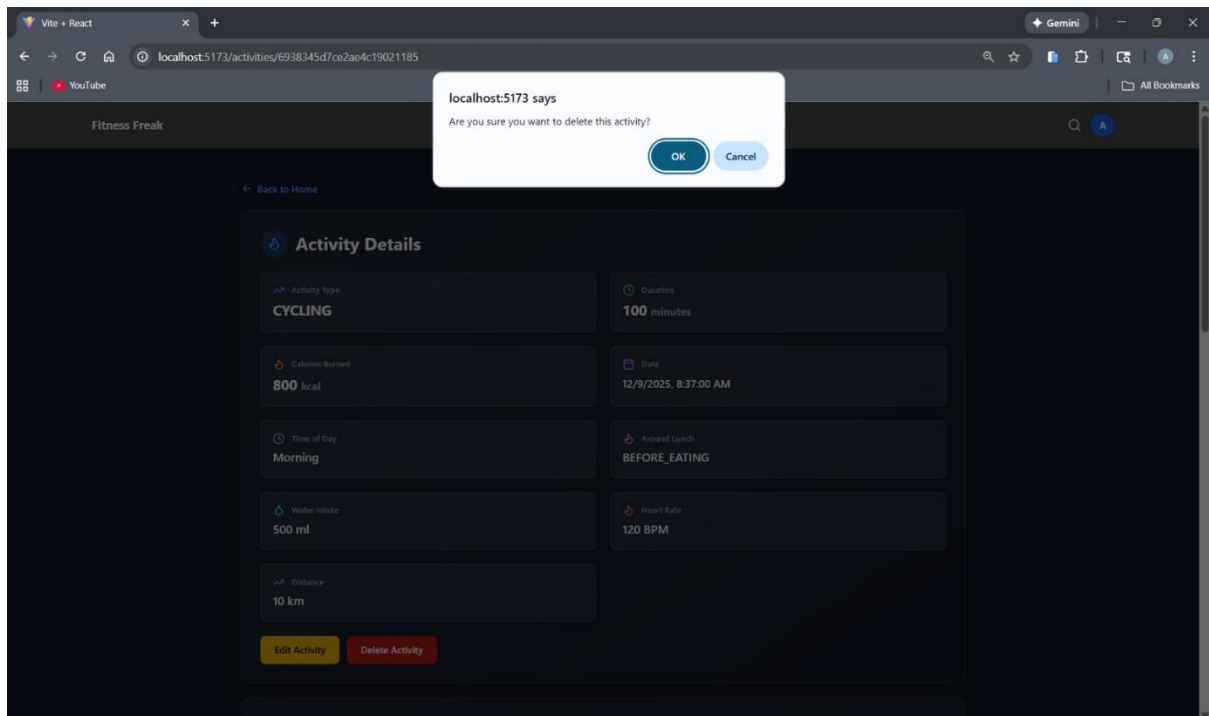
The "AI Recommendation" section provides an analysis and improvements for the activity:

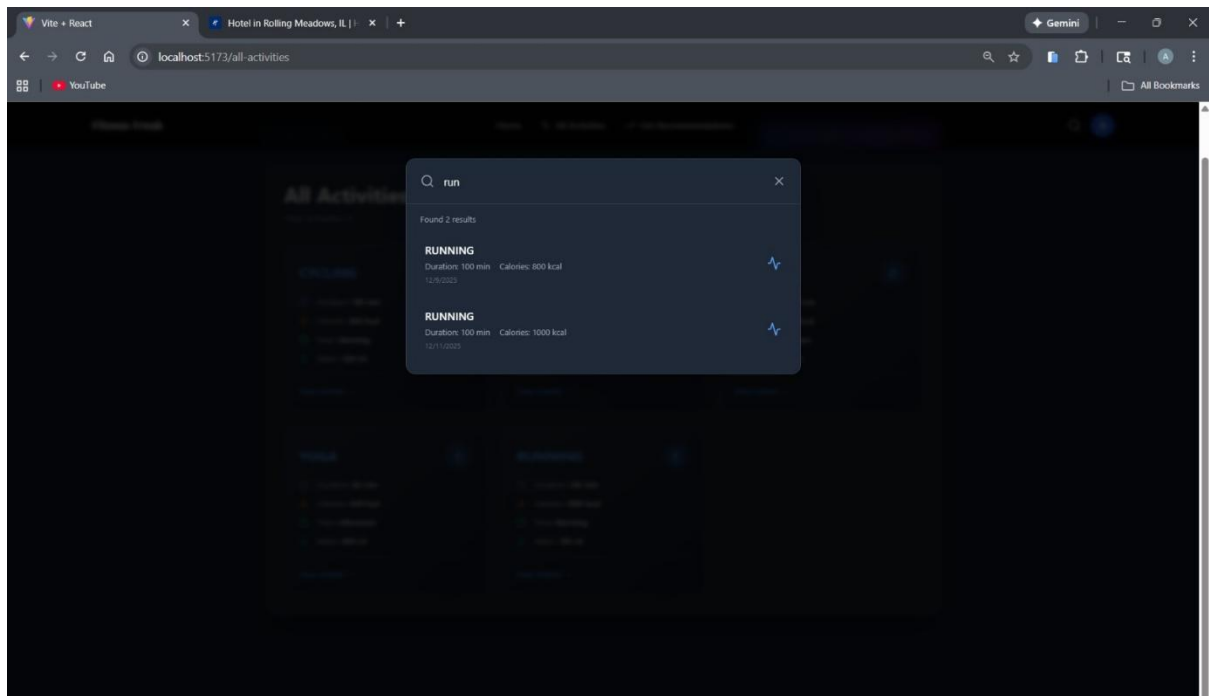
Analysis

- Overall:** The cycling activity shows a steady, low-intensity effort over a relatively long duration. Calorie burn seems decent for the average heart rate, but the distance covered is relatively low for the duration, suggesting a slow pace, potentially due to challenging terrain or low effort.
- Pace:** The pace is quite slow, covering only 10 km in 100 minutes, resulting in an average speed of 6 km/h. This could indicate a very casual ride, significant elevation changes, or headwinds.
- Heart Rate:** The average heart rate of 120 BPM indicates a low-intensity effort. Without knowing the user's age and maximum heart rate, it's difficult to determine specific heart rate zones. However, this heart rate likely places the user in the low to moderate intensity range, suitable for a recovery ride or active recovery.
- Distance:** The total distance of 10 km is low for the duration of 100 minutes. This suggests either a very low effort or a route with significant obstacles or elevation changes. It indicates room for improvement in either speed or route selection.
- Calories:** Burning 800 calories in 100 minutes is a reasonable amount, given the low-intensity effort as indicated by the heart rate. However, this could be optimized by increasing the intensity and thus covering a greater distance.

Improvements

- Pace/Speed:** Increase average speed by incorporating interval training and varying terrain. Try increasing cadence to improve efficiency and power output.
- Distance:** Aim to increase the distance covered within the same duration. This can be achieved by increasing speed, choosing a flatter route, or improving overall fitness and endurance.





REFERENCES

- [1] Google. (2024). *Gemini API documentation*. <https://ai.google.dev>
- Pivotal Software. (2024). *Spring Boot reference documentation*.
<https://spring.io/projects/spring-boot>
- [2] Apache Software Foundation. (2024). *Apache Kafka developer guide*.
<https://kafka.apache.org/documentation/>
- [3] Red Hat. (2024). *Keycloak documentation: Identity and access management*.
<https://www.keycloak.org/documentation>
- [4] React Team. (2024). *React official documentation*. <https://react.dev>
- [5] Redux Team. (2024). *Redux Toolkit documentation*. <https://redux-toolkit.js.org>
- [6] PostgreSQL Global Development Group. (2024). *PostgreSQL database documentation*.
<https://www.postgresql.org/docs/>
- [7] MongoDB Inc. (2024). *MongoDB documentation*. <https://www.mongodb.com/docs/>
- [8] IEEE Computer Society. (2020). *ACM/IEEE Software Engineering Code of Ethics*.
Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.\
- [9] Newman, S. (2019). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- [10] Verma, A. (2022). Event-Driven Architecture in Modern Software Systems. *Journal of Distributed Computing Research*, 45(3), 112–128.